

ARTIZONE REST API SPECIFICATION

Version 1.0 | Base URL: <https://api.artizonespa.com/v1>

Authentication: JWT Bearer token for admin endpoints

Content-Type: application/json for all requests

AUTHENTICATION

POST /auth/login

Staff login for admin dashboard access.

Request:

```
{
  "email": "reception@artizonespa.com",
  "password": "secure_password"
}
```

Response 200:

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": {
    "id": "uuid",
    "name": "Maya",
    "role": "esthetician",
    "email": "reception@artizonespa.com"
  },
  "expires_at": "2026-06-09T09:48:00Z"
}
```

Response 401:

```
{
  "error": "Invalid credentials",
  "code": "AUTH_INVALID"
}
```

Headers for all admin endpoints:

```
Authorization: Bearer <token>
Content-Type: application/json
```

PUBLIC ENDPOINTS (No auth required)

GET /services

Returns active services for the booking form dropdown.

Response 200:

```
{
  "services": [
    {
      "id": "svc_001",
      "name": "Hydrafacial",
      "category": "skin",
      "duration_min": 60,
      "buffer_min": 10,
      "price": 45.00,
      "required_capability": null
    },
    {
      "id": "svc_004",
      "name": "Laser session",
      "category": "laser",
      "duration_min": 20,
      "buffer_min": 10,
      "price": 35.00,
      "required_capability": "laser"
    }
  ]
}
```

POST /booking

Customer submits a booking request. Creates appointment with status='requested'.

Request:

```
{
  "name": "Rana Haddad",
  "phone": "0790412758",
  "whatsapp_optin": true,
  "service_id": "svc_001",
  "preferred_date": "2026-06-15",
  "preferred_time": "14:00",
  "notes": "First visit, sensitive skin",
  "source": "web_form"
}
```

Validation Rules:

- phone: Must match Jordan mobile format (07XXXXXXXX or +9627XXXXXXXX)
- preferred_date: Must be today or future, within 90 days
- preferred_time: Must be within working hours (10:00–19:00)
- service_id: Must be active service
- source: One of 'web_form' | 'whatsapp' | 'phone' | 'walk_in'

Response 201:

```
{
  "success": true,
  "appointment_id": "appt_001",
  "status": "requested",
  "message": "Request received. We'll confirm via WhatsApp shortly.",
  "estimated_confirmation": "within 2 hours"
}
```

Response 400 (validation error):

```
{
  "error": "Validation failed",
  "code": "VALIDATION_ERROR",
  "details": [
    { "field": "phone", "message": "Invalid Jordan phone format" },
    { "field": "preferred_date", "message": "Date must be within 90 days" }
  ]
}
```

Response 429 (rate limit):

```
{
  "error": "Too many requests",
  "code": "RATE_LIMITED",
  "retry_after": 3600
}
```

Backend Logic:

1. Normalize phone to E.164: +962790412758
2. Create customer (if new) or find existing by phone
3. Calculate `end_at = start_at + duration_min + buffer_min`
4. Create appointment with `status='requested'`, `staff_id=null`, `resource_id=null`
5. Log event: `requested → requested` (initial creation)
6. Return `appointment_id`

ADMIN ENDPOINTS (JWT required)

GET /admin/dashboard

Returns daily pulse metrics + pending queue + today's schedule.

Query params:

- `date` (optional): ISO date string, defaults to today

Response 200:

```

{
  "date": "2026-06-15",
  "metrics": {
    "today_total": 12,
    "pending": 5,
    "confirmed": 7,
    "no_shows": 1,
    "completed": 3
  },
  "pending_requests": [
    {
      "id": "appt_001",
      "customer": {
        "id": "cust_001",
        "name": "Rana Haddad",
        "phone": "+962790412758",
        "initials": "RH",
        "is_returning": true,
        "past_visits": 3
      },
      "service": {
        "id": "svc_001",
        "name": "Hydrafacial",
        "duration_min": 60
      },
      "requested_time": "2026-06-15T14:00:00+03:00",
      "source": "whatsapp",
      "notes": "First visit, sensitive skin",
      "created_at": "2026-06-14T10:30:00Z",
      "wait_time_hours": 18
    }
  ],
  "todays_schedule": [
    {
      "time": "10:00",
      "customer_name": "Lina K.",
      "service": "Facial",
      "therapist": "Maya",
      "room": "Treatment room 2",
      "status": "confirmed",
      "duration_min": 60
    },
    {
      "time": "11:30",
      "customer_name": "Sara A.",
      "service": "Laser",
      "therapist": "Dr. Nour",
      "room": "Laser room",
      "status": "confirmed",
      "duration_min": 20
    }
  ]
}

```

```
}  
]  
}
```

GET /admin/appointments/:id

Full request/appointment detail.

Response 200 (pending request):

```
{  
  "id": "appt_001",  
  "status": "requested",  
  "customer": {  
    "id": "cust_001",  
    "name": "Rana Haddad",  
    "phone": "+962790412758",  
    "whatsapp_optin": true,  
    "initials": "RH",  
    "is_returning": true,  
    "past_visits": 3,  
    "created_at": "2026-05-01T08:00:00Z"  
  },  
  "service": {  
    "id": "svc_001",  
    "name": "Hydrafacial",  
    "category": "skin",  
    "duration_min": 60,  
    "buffer_min": 10,  
    "price": 45.00  
  },  
  "requested": {  
    "date": "2026-06-15",  
    "time": "14:00",  
    "local": "Tue 10 Jun, 2:00 PM"  
  },  
  "assigned": {  
    "staff_id": null,  
    "staff_name": null,  
    "resource_id": null,  
    "resource_name": null,  
    "start_at": null,  
    "end_at": null  
  },  
  "source": "whatsapp",  
  "notes": "First visit, sensitive skin",  
  "history": [  
    {  
      "from_status": null,  
      "to_status": "requested",  
    }  
  ]  
}
```

```
    "actor": "system",
    "created_at": "2026-06-14T10:30:00Z"
  }
]
```

Response 200 (confirmed appointment):

```
{
  "id": "appt_002",
  "status": "confirmed",
  "customer": { "..."},
  "service": { "..."},
  "assigned": {
    "staff_id": "st_001",
    "staff_name": "Maya",
    "resource_id": "res_002",
    "resource_name": "Treatment room 2",
    "start_at": "2026-06-15T14:00:00+03:00",
    "end_at": "2026-06-15T15:10:00+03:00"
  },
  "source": "web_form",
  "confirmed_at": "2026-06-14T11:00:00Z",
  "history": [
    { "from_status": null, "to_status": "requested", "actor": "system", "..."},
    { "from_status": "requested", "to_status": "confirmed", "actor": "Maya", "..."}
  ]
}
```

Response 404:

```
{
  "error": "Appointment not found",
  "code": "NOT_FOUND"
}
```

PUT /admin/appointments/:id/confirm

The core transaction. Assigns staff + room + final times, flips status to ‘confirmed’.

Request:

```
{
  "staff_id": "st_001",
  "resource_id": "res_002",
  "start_at": "2026-06-15T14:00:00+03:00",
  "notes": "Confirmed by Maya at reception"
}
```

Validation (4-gate check):

1. **Gate 1 — Skill match:** Staff must have skill matching service.category (or staff_service override)
2. **Gate 2 — Working hours:** Staff must be working at start_at weekday/time per staff_availability
3. **Gate 3 — Staff free:** No confirmed appointment overlap for this staff_id (exclusion constraint enforces this)
4. **Gate 4 — Room free:** No confirmed appointment overlap for this resource_id (exclusion constraint enforces this)

Response 200:

```
{
  "success": true,
  "appointment_id": "appt_001",
  "status": "confirmed",
  "assigned": {
    "therapist": "Maya",
    "room": "Treatment room 2",
    "start_at": "2026-06-15T14:00:00+03:00",
    "end_at": "2026-06-15T15:10:00+03:00"
  },
  "whatsapp_sent": true,
  "message": "Booking confirmed and WhatsApp notification sent"
}
```

Response 409 (conflict — double booking blocked):

```
{
  "error": "Slot no longer available",
  "code": "SLOT_CONFLICT",
  "details": {
    "staff_available": false,
    "room_available": true,
    "conflicting_appointment": "appt_003"
  }
}
```

Response 422 (skill mismatch):

```
{
  "error": "Staff cannot perform this service",
  "code": "SKILL_MISMATCH",
  "details": {
    "service_category": "laser",
    "staff_skills": ["skin", "body"]
  }
}
```

Backend Transaction (must be atomic):

```
BEGIN;
-- 1. Update appointment with staff_id, resource_id, start_at, end_at, status='confirmed',
↪ confirmed_at=now()
```

```
-- 2. Insert appointment_event (requested → confirmed, actor=current_user)
-- 3. Exclusion constraint validates no overlap
-- 4. If all pass, COMMIT and trigger WhatsApp confirmation
-- 5. If exclusion constraint fails, ROLLBACK with 409
COMMIT;
```

PUT /admin/appointments/:id/decline

Decline a pending request.

Request:

```
{
  "reason": "Slot fully booked, offered alternative times"
}
```

Response 200:

```
{
  "success": true,
  "appointment_id": "appt_001",
  "status": "declined",
  "whatsapp_sent": true
}
```

PUT /admin/appointments/:id/reschedule

Edit a confirmed appointment (change time, staff, or room).

Request:

```
{
  "staff_id": "st_002",
  "resource_id": "res_001",
  "start_at": "2026-06-15T16:00:00+03:00"
}
```

Response 200:

```
{
  "success": true,
  "appointment_id": "appt_001",
  "status": "confirmed",
  "previous": {
    "start_at": "2026-06-15T14:00:00+03:00",
    "staff_id": "st_001",
    "resource_id": "res_002"
  },
  "new": {
    "start_at": "2026-06-15T16:00:00+03:00",

```

```
{
  "staff_id": "st_002",
  "resource_id": "res_001"
},
"whatsapp_sent": true
}
```

Note: Rescheduling runs the same 4-gate check + exclusion constraints. The old slot is released, new slot is reserved atomically.

GET /admin/therapists-available

Query available therapists for a service at a specific time.

Query params:

- `service_id` (required): UUID of service
- `date` (required): ISO date (e.g., 2026-06-15)
- `time` (required): HH:MM (e.g., 14:00)
- `exclude_appointment_id` (optional): Skip self when rescheduling

Response 200:

```
{
  "service_id": "svc_001",
  "requested": "2026-06-15T14:00:00+03:00",
  "duration_min": 60,
  "buffer_min": 10,
  "available_therapists": [
    {
      "id": "st_001",
      "name": "Maya",
      "role": "esthetician",
      "skills": ["skin", "body"],
      "is_free": true,
      "working_today": true
    },
    {
      "id": "st_002",
      "name": "Lara",
      "role": "stylist",
      "skills": ["skin", "other"],
      "is_free": true,
      "working_today": true
    }
  ],
  "unavailable_therapists": [
    {
      "id": "st_003",
      "name": "Dr. Nour",
      "reason": "skill_mismatch",

```

```
    "has_skill": false
  }
]
```

SQL Logic (4-gate check):

```
SELECT s.id, s.name, s.role
FROM staff s
JOIN staff_skill sk ON sk.staff_id = s.id
JOIN service svc ON svc.id = :service_id
WHERE sk.skill = svc.category
  AND s.active = true
  AND EXISTS (
    SELECT 1 FROM staff_availability wa
    WHERE wa.staff_id = s.id
      AND wa.weekday = EXTRACT(DOW FROM :date::date)
      AND wa.start_time <= :slot_start_time
      AND wa.end_time >= :slot_end_time
  )
  AND NOT EXISTS (
    SELECT 1 FROM appointment a
    WHERE a.staff_id = s.id
      AND a.status = 'confirmed'
      AND a.id != :exclude_appointment_id
      AND a.start_at < :new_end
      AND a.end_at > :new_start
  )
  AND NOT EXISTS (
    SELECT 1 FROM staff_time_off t
    WHERE t.staff_id = s.id
      AND t.start_at < :new_end
      AND t.end_at > :new_start
  );
```

GET /admin/rooms-available

Query available rooms for a service at a specific time.

Query params:

- service_id (required): UUID
- start_at (required): ISO datetime with timezone
- end_at (required): ISO datetime with timezone
- exclude_appointment_id (optional)

Response 200:

```
{
  "service_id": "svc_004",
```

```

"requested": "2026-06-15T14:00:00+03:00",
"available_rooms": [
  {
    "id": "res_003",
    "name": "Laser room",
    "type": "machine",
    "capabilities": ["laser"],
    "is_free": true
  }
],
"unavailable_rooms": [
  {
    "id": "res_001",
    "name": "Treatment room 1",
    "reason": "capability_mismatch",
    "has_capability": false
  },
  {
    "id": "res_002",
    "name": "Treatment room 2",
    "reason": "occupied",
    "conflicting_appointment": "appt_005"
  }
]
}

```

GET /admin/calendar/day

Day view calendar data — one column per practitioner.

Query params:

- date (required): ISO date

Response 200:

```

{
  "date": "2026-06-15",
  "hours": ["10:00", "10:30", "11:00", "11:30", "12:00", "12:30", "..."],
  "columns": [
    {
      "staff_id": "st_001",
      "staff_name": "Maya",
      "role": "esthetician",
      "working_hours": { "start": "10:00", "end": "19:00" },
      "blocks": [
        {
          "type": "appointment",
          "appointment_id": "appt_002",
          "customer_name": "Lina K.",
          "service": "Facial",

```

```

        "start_at": "10:00",
        "end_at": "11:10",
        "duration_min": 60,
        "status": "confirmed",
        "color": "#1D9E75"
    },
    {
        "type": "break",
        "start_at": "12:00",
        "end_at": "13:00",
        "reason": "Lunch"
    }
]
}
]
}

```

REPORTING ENDPOINTS (Phase 2+)

GET /admin/reports/demand

Flexible demand reporting endpoint.

Query params:

- from (required): ISO date
- to (required): ISO date
- group_by (required): day_hour | service | channel | therapist
- status (optional): requested | confirmed — defaults to 'requested'
- therapist_id (optional): filter by staff
- channel (optional): filter by source

Response 200 (group_by=day_hour):

```

{
  "group_by": "day_hour",
  "status": "requested",
  "from": "2026-05-08",
  "to": "2026-06-08",
  "buckets": [
    { "bucket_key": [6, 14], "requests": 12 }, // Sat, 2pm
    { "bucket_key": [6, 15], "requests": 8 },  // Sat, 3pm
    { "bucket_key": [0, 10], "requests": 3 }   // Sun, 10am
  ]
}

```

Response 200 (group_by=service):

```

{

```

```
"group_by": "service",
"buckets": [
  { "bucket_key": "Massage", "requests": 142 },
  { "bucket_key": "Hydrafacial", "requests": 118 },
  { "bucket_key": "Laser", "requests": 64 }
]
}
```

WAITLIST ENDPOINTS (Phase 2+)

POST /admin/waitlist

Add customer to waitlist for a preferred slot.

Request:

```
{
  "customer_id": "cust_001",
  "service_id": "svc_001",
  "staff_id": "st_001",           // optional preferred therapist
  "preferred_from": "2026-06-15T14:00:00+03:00",
  "preferred_to": "2026-06-15T19:00:00+03:00"
}
```

Response 201:

```
{
  "waitlist_id": "wl_001",
  "status": "waiting",
  "position": 3,
  "estimated_wait": "2-3 days"
}
```

POST /admin/waitlist/:id/offer

Offer a freed slot to a waitlisted customer.

Request:

```
{
  "appointment_id": "appt_new_001", // the held tentative appointment
  "offer_ttl_min": 20
}
```

Response 200:

```
{
  "success": true,
  "waitlist_id": "wl_001",
  "status": "offered",
}
```

```
"whatsapp_sent": true,
"expires_at": "2026-06-15T14:20:00+03:00"
}
```

PACKAGE ENDPOINTS (Phase 2+)

GET /admin/customers/:id/packages

Get customer's active packages.

Response 200:

```
{
  "customer_id": "cust_001",
  "packages": [
    {
      "id": "cp_001",
      "package_name": "Laser — 6 sessions",
      "kind": "package",
      "sessions_left": 3,
      "total_sessions": 6,
      "expires_at": "2026-08-12T00:00:00Z",
      "status": "active"
    },
    {
      "id": "cp_002",
      "package_name": "Monthly facial",
      "kind": "membership",
      "sessions_left": null,
      "expires_at": "2026-07-15T00:00:00Z",
      "status": "active"
    }
  ]
}
```

ERROR CODE REFERENCE

HTTP	Code	When
400	VALIDATION_ERROR	Missing/invalid fields
401	AUTH_INVALID	Bad login or expired token
403	FORBIDDEN	Valid auth but insufficient role
404	NOT_FOUND	Resource doesn't exist
409	SLOT_CONFLICT	Double-booking blocked by exclusion constraint
422	SKILL_MISMATCH	Staff can't perform service
422	CAPABILITY_MISMATCH	Room lacks required machine
429	RATE_LIMITED	Too many requests

Table 1 – continued

HTTP	Code	When
500	INTERNAL_ERROR	Server error (log and report)

RATE LIMITING

Endpoint	Limit
POST /booking	5 requests/hour per IP + phone
POST /auth/login	10 attempts/hour per IP
All admin GET	100 requests/minute per token
All admin PUT/POST	30 requests/minute per token

WEBHOOKS (Inbound from Meta/WhatsApp)

POST /webhooks/whatsapp

Handles incoming WhatsApp messages and button taps.

Verification (GET):

```
GET /webhooks/whatsapp?hub.verify_token=YOUR_VERIFY_TOKEN&hub.challenge=CHALLENGE_STRING
Response: hub.challenge (200) or 403
```

Incoming message (POST):

```
{
  "object": "whatsapp_business_account",
  "entry": [{
    "changes": [{
      "value": {
        "messages": [{
          "from": "962790412758",
          "button": {
            "payload": "ACCEPT_w1_001",
            "text": "Accept"
          }
        }]
      }
    }]
  }]
}
```

Response: Always return 200 immediately, then process async.

ENVIRONMENT VARIABLES

```
# Database
DATABASE_URL=postgresql://user:pass@host:5432/artizone

# JWT
JWT_SECRET=your_256_bit_secret_here
JWT_EXPIRES_HOURS=24

# WhatsApp Cloud API
WHATSAPP_TOKEN=EAAB...
PHONE_NUMBER_ID=123456789
VERIFY_TOKEN=your_webhook_verify_token

# Server
PORT=3000
NODE_ENV=production
TIMEZONE=Asia/Amman

# Frontend
ADMIN_URL=https://admin.artizonespa.com
PUBLIC_WIDGET_URL=https://api.artizonespa.com/v1/booking-widget.js
```